

LAPORAN PRAKTIKUM STRUKTUR DATA

Bubble, Selection dan Insertion Sorting



Oleh :

MUHAMMAD ARIF

NIM 2511532017

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 19 Mei 2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadirat Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, laporan praktikum **“Bubble, Selection dan Insertion Sorting”** ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai dokumentasi dan pemahaman hasil kegiatan praktikum yang bertujuan untuk memahami bagaimana konsep dari Bubble, Selection dan Insertion Sorting. Penulis juga terbuka untuk menerima saran dan kritik guna perbaikan di masa mendatang.

Penulis mengucapkan terima kasih sebesar-besarnya kepada:

1. DR. Wahyudi, S.T, M.T selaku pembimbing mata kuliah struktur data, yang telah memberikan bimbingan, ilmu, dan motivasi selama proses pembelajaran.

Padang, 19 Mei 2026

Muhammad Arif

DAFTAR PUSTAKA

LAPORAN PRAKTIKUM STRUKTUR DATA	i
KATA PENGANTAR.....	ii
DAFTAR PUSTAKA.....	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	2
BAB II PEMBAHASAN	3
2.1 Landasan teori	3
2.2 Langkah Pengerjaan	4
BAB III KESIMPULAN.....	26
DAFTAR PUSTAKA.....	27

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam era digital sekarang, pengelolaan dan pengolahan data menjadi fondasi utama dalam pengembangan sistem komputasi modern. Salah satu operasi yang penting dalam pengolahan data adalah pengurutan (sorting), yang berperan penting dalam mengorganisir kumpulan data agar lebih mudah dicari, dianalisis dan diproses oleh algoritma lain.

Sorting memiliki beragam pengurutan dengan kompleksitas waktu dan ruang yang beragam seperti BubbleSort, InsertionSort, MergeSort, QuickSort, dan SelectionSort. Setiap algoritma memiliki profil performa yang berbeda tergantung pada ukuran data, tingkat keterurutan awal, kebutuhan akan stabilitas pengurutan, serta batasan memori. Pemilihan algoritma yang tepat tidak hanya bergantung pada teori, tetapi juga pada konteks penggunaan nyata dan karakteristik data yang diolah.

Oleh karena itu, laporan praktikum ini dibuat sebagai dokumentasi sistematis dari proses implementasi, pengujian dan analisis algoritma sorting. Laporan ini bertujuan untuk memperkuat kompetensi praktikan dalam merancang solusi komputasi yang efisien.

1.2 Tujuan

Tujuan dari dilakukannya praktikum ini adalah

1. Memahami struktur Algoritma Bubble, Selection dan Insertion Sorting
2. Memahami langkah-langkah menulis Bubble, Selection dan Insertion Sorting

1.3 Manfaat

Manfaat yang diperoleh dari praktikum ini adalah

1. Meningkatkan pemahaman tentang Bubble, Selection dan Insertion Sorting
2. Meningkatkan pemahaman bagaimana cara menulis Bubble, Selection dan Insertion Sorting

BAB II

PEMBAHASAN

2.1 Landasan teori

Sorting merupakan ilmu komputer yang bertujuan untuk menyusun menyusun ulang elemen-elemen dalam suatu koleksi data berdasarkan nilai kunci tertentu, umumnya dalam urutan menaik (ascending) atau menurun (descending). Proses ini tidak hanya mempermudah pencarian data, tetapi juga menjadi prasyarat bagi banyak algoritma lain seperti pencarian biner, penggabungan data (merge), dan kompresi.

Bubble Sort adalah algoritma pengurutan sederhana yang bekerja dengan cara berulang kali membandingkan dua elemen berdekatan dan menukar posisinya jika keduanya berada dalam urutan yang salah. Proses ini dilakukan secara iteratif hingga tidak ada lagi pertukaran yang diperlukan, yang menandakan bahwa data telah terurut sepenuhnya. Nama "bubble" berasal dari cara elemen dengan nilai ekstrem secara bertahap "mengapung" ke posisi akhirnya pada setiap iterasi. Bubble Sort memiliki kompleksitas waktu terburuk (worst-case) dan rata-rata (average-case) sebesar $O(n^2)$, di mana n adalah jumlah elemen. Algoritma ini bersifat *stable* dan *in-place* karena hanya membutuhkan memori tambahan konstan $O(1)$ untuk operasi penukaran [1]. Meskipun mudah dipahami dan diimplementasikan, Bubble Sort umumnya tidak direkomendasikan untuk dataset besar karena jumlah perbandingan dan penukaran yang sangat tinggi.

Insertion Sort bekerja dengan prinsip membangun array terurut secara bertahap. Algoritma ini memproses elemen satu per satu dari kiri ke kanan, lalu menyisipkan elemen yang sedang diproses ke posisi yang tepat di dalam bagian array yang sudah terurut. Analogi yang sering digunakan adalah cara seseorang mengurutkan kartu remi di tangan: mengambil kartu baru dan menyelipkannya di antara kartu-kartu yang sudah tersusun rapi. Seperti halnya Bubble Sort, Insertion Sort memiliki kompleksitas waktu terburuk dan rata-rata sebesar $O(n^2)$, serta kompleksitas terbaik $O(n)$ untuk data yang hampir atau sudah terurut. Algoritma ini juga bersifat *stable* dan *in-place* dengan kebutuhan ruang tambahan $O(1)$. Keunggulan utama Insertion Sort terletak pada adaptabilitasnya terhadap data kecil atau data yang sudah sebagian terurut, serta jumlah operasi penulisan/penyisipan yang relatif rendah.

Selection Sort bekerja dengan prinsip mencari elemen dengan nilai terkecil (atau terbesar) dari bagian array yang belum terurut, kemudian menukarnya dengan elemen pada posisi pertama dari bagian tersebut. Proses ini diulang secara iteratif hingga seluruh elemen berada pada posisi yang benar. Berbeda dengan Bubble Sort yang melakukan penukaran secara bertahap antar elemen berdekatan, Selection Sort meminimalkan jumlah operasi penukaran karena hanya melakukan maksimal satu *swap* per iterasi. Dari segi kompleksitas, Selection Sort memiliki waktu eksekusi $O(n^2)$ untuk kasus terbaik, rata-rata, maupun terburuk, karena jumlah perbandingan selalu tetap sebanyak $\frac{n(n-1)}{2}$ terlepas dari kondisi awal data. Algoritma ini bersifat in-place dengan kebutuhan memori tambahan $O(1)$. Namun, Selection Sort umumnya tidak bersifat stable karena proses penukaran dapat mengubah urutan relatif elemen yang bernilai sama.

2.2 Langkah Pengerjaan

- InsertionSort_2511532017
 1. Buka eclipse dan buat package baru dengan nama pekan7_2511532017 dan class InsertionSort_2511532017.
 2. Setelah dibuat, kita akan membuat algoritma InsertionSort

```
package pekan7_2511532017;

public class InsertionSort_2511532017 {
    public static void InsertionSort_2511532017 (int[] arr) {
        int n_2017 = arr.length;
        for (int i_2017 = 1; i_2017 < n_2017; i_2017++) {
            int key = arr[i_2017];
            int j_2017 = i_2017 - 1;
            while (j_2017 >= 0 && arr[j_2017] > key) {
                arr[j_2017 + 1] = arr[j_2017];
                j_2017--;
            }
            arr[j_2017 + 1] = key;
        }
    }
}
```

3. int n_2017 = arr.length; menyimpan jumlah elemen array, perulangan dimulai dari index 1 karena elemen diindex 0 dianggap sudah terurut secara trivial. int key = arr[i_2017]; mengambil elemen yang akan disisipkan ke posisi yang benar. int j_2017 = i_2017 - 1; ini sebagai petunjuk ke elemen sebelumnya (wilayah array yang sudah terurut)

```

package pekan7_2511532017;

public class InsertionSort_2511532017 {
    public static void InsertionSort_2511532017 (int[] arr) {
        int n_2017 = arr.length;
        for (int i_2017 = 1; i_2017 < n_2017; i_2017++) {
            int key = arr[i_2017];
            int j_2017 = i_2017 - 1;
            while (j_2017 >= 0 && arr[j_2017] > key) {
                arr[j_2017 + 1] = arr[j_2017];
                j_2017--;
            }
            arr[j_2017 + 1] = key;
        }
    }
}

```

4. while (j_2017 >= 0 && arr[j_2017] > key) selama j masih dalam batas array dan kiri arr[j_2017] lebih besar dari key maka elemen tersebut digeser ke kanan arr[j+1] = arr[j], lalu j dikurangi. arr[j_2017 + 1] = key; setelah loop while berhenti (ditemukan elemen yang <= key atau sudah di awal array) key disisipkan ke posisi kosong yang tepat.

```

7 public static void main (String [] args) {
8     int arr_2017[] = {23, 78, 45, 8, 32, 56, 1};
9     int n_2017 = arr_2017.length;
10    System.out.printf("array yang belum terurut:\n");
11    for (int i_2017 = 0; i_2017 < n_2017; i_2017++)
12        System.out.print(arr_2017[i_2017] + " ");
13    System.out.println("");
14    InsertionSort_2511532017 (arr_2017);
15    System.out.printf("array yang terurut:\n");
16    for (int i = 0; i < n_2017 ; i++)
17        System.out.print(arr_2017[i] + " ");
18    System.out.println("");
19 }
20 }

```

5. Method main, inialisasi dengan data {23, 78, 45, 8, 32, 56, 1}. Cetak sebelum sort menggunakan loop for dan System.out.print() untuk menampilkan elemen dengan spasi. Pemanggilan method InsertionSort_2511532017(arr_2017); cetak setelah terurut.
6. Output

```

array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78

```

- SelectionSort_2511532017
 1. Kita akan membuat class baru untuk SelectionSort_2511532017


```

1 package pekan7_2511532017;
2
3 public class SelectionSort_2511532017 {
4     public static void SelectionSort_2511532017 (int[] arr_2017) {
5         int n_2017 = arr_2017.length;
6         for (int i_2017=0; i_2017 < n_2017 ; i_2017++) {
7             int minIndex_2017 = i_2017;
8             for (int j_2017=i_2017 + 1; j_2017 < n_2017; j_2017++) {
9                 if (arr_2017[j_2017] < arr_2017[minIndex_2017]) {
10                     minIndex_2017 = j_2017;
11                 }
12             }
13             int temp_2017 = arr_2017[i_2017];
14             arr_2017[i_2017] = arr_2017[minIndex_2017];
15             arr_2017[minIndex_2017] = temp_2017;
16         }
17     }
18 }

```

2. `int n_2017 = arr_2017.length;` menyimpan jumlah elemen array, loop luar akan menandai posisi yang akan diisi oleh elemen terkecil dari bagian array yang belum terurut setiap iterasi bertambah 1 (wilayah terurut di sebelah kiri semakin panjang).

```

1 package pekan7_2511532017;
2
3 public class SelectionSort_2511532017 {
4     public static void SelectionSort_2511532017 (int[] arr_2017) {
5         int n_2017 = arr_2017.length;
6         for (int i_2017=0; i_2017 < n_2017 ; i_2017++) {
7             int minIndex_2017 = i_2017;
8             for (int j_2017=i_2017 + 1; j_2017 < n_2017; j_2017++) {
9                 if (arr_2017[j_2017] < arr_2017[minIndex_2017]) {
10                     minIndex_2017 = j_2017;
11                 }
12             }
13             int temp_2017 = arr_2017[i_2017];
14             arr_2017[i_2017] = arr_2017[minIndex_2017];
15             arr_2017[minIndex_2017] = temp_2017;
16         }
17     }
18 }

```

3. `int minIndex_2017 = i_2017;`, elemen di indek i adalah yang paling kecil di sisa array, loop dalam akan memindai semua elemen di sebelah kanan I untuk mencari nilai yang lebih kecil dari `arr[minIndex]`. `if (arr_2017[j_2017] < arr_2017[minIndex_2017])` jika ditemukan elemen lebih kecil, update `minIndex_2017` ke indek j

```

1 package pekan7_2511532017;
2
3 public class SelectionSort_2511532017 {
4     public static void SelectionSort_2511532017 (int[] arr_2017) {
5         int n_2017 = arr_2017.length;
6         for (int i_2017=0; i_2017 < n_2017 ; i_2017++) {
7             int minIndex_2017 = i_2017;
8             for (int j_2017=i_2017 + 1; j_2017 < n_2017; j_2017++) {
9                 if (arr_2017[j_2017] < arr_2017[minIndex_2017]) {
10                     minIndex_2017 = j_2017;
11                 }
12             }
13             int temp_2017 = arr_2017[i_2017];
14             arr_2017[i_2017] = arr_2017[minIndex_2017];
15             arr_2017[minIndex_2017] = temp_2017;
16         }
17     }
18 }

```

4. setelah loop selesai `minIndex_2017` berisi index elemen terkecil di bagian terurut dan elemen tersebut ditukar dengan elemen di posisi `i_2017` menggunakan variabel sementara `temp_2017`.

```

8 public static void main (String[] args) {
9     int arr[] = {23, 78, 45, 8, 32, 56, 1};
10    int n = arr.length;
11    System.out.printf("array yang belum terurut :\n");
12    for (int i =0; i<n; i++)
13        System.out.print(arr[i] + " ");
14    System.out.println("");
15    SelectionSort_2511532017(arr);
16    System.out.printf("array yang terurut :\n");
17    for (int i =0; i<n; i++)
18        System.out.print(arr[i] + " ");
19    System.out.println("");
20 }
21 }
22 }
23 }

```

5. Method main, array contoh dengan data {23, 78, 45, 8, 32, 56, 1} dan panjangnya disimpan di n, cetak sebelum sort dengan loop, pemanggilan method SelectionSort_2511532017(arr);, dan cetak setelah sort dengan loop
6. Output

```

<terminated> SelectionSort_2511532017
array yang belum terurut :
23 78 45 8 32 56 1
array yang terurut :
1 8 23 32 45 56 78

```

- BubleSort_2511532017

1. kita akan membuat class baru untuk BubleSort_2511532017

```

package pekan7_2511532017;

public class BubleSort_2511532017 {
    public static void BubleSort_2511532017(int[] arr_2017) {
        int n_2017 = arr_2017.length;
        for (int i_2017 =0; i_2017 < n_2017; i_2017++) {
            for (int j_2017 = 0; j_2017< n_2017-i_2017-1; j_2017++) {
                if(arr_2017[j_2017] > arr_2017[j_2017 +1]) {
                    int temp = arr_2017[j_2017];
                    arr_2017[j_2017] = arr_2017[j_2017+1];
                    arr_2017[j_2017+1]= temp;
                    //system.out.println("data;" +arr[j]+arr[j+1]);
                }
            }
        }
    }
}

```

2. int n_2017 = arr_2017.length; untuk menyimpan jumlah elemen array, loop luar untuk mengontrol jumlah pass dan elemen terbesar ke posisi paling kanan. Loop dalam membandingkan pasangan

elemen yang bersebelahan. Jika elemen kiri lebih besar dari kanan lakukan pertukaran menggunakan variabel sementara temp

```
public static void main(String[] args) {
    int arr[] = {23, 78, 45, 8, 32, 56, 1};
    int n = arr.length;
    System.out.print("array yang belum terurut:");
    for (int i = 0; i < n; i++)
        System.out.print(arr[i] + " ");
    System.out.println("");
    BubbleSort_2511532017(arr);
    System.out.print("array yang terurut:");
    for (int i = 0; i < n; i++)
        System.out.print(arr[i] + " ");
    System.out.println("");
}
```

3. method main inialisasi array {23, 78, 45, 8, 32, 56, 1}, cetak awal sebelum sort dengan loop. Pemanggilan method BubbleSort_2511532017(arr);, cetak akhir dengan method
4. output

```
array yang belum terurut:23 78 45 8 32 56 1
array yang terurut:1 8 23 32 45 56 78
```

- InsertionGUI_2511532017

1. Buat class dengan nama InsertionGUI_2511532017

```
1 package pekan7_2511532017;
2
3 import java.awt.BorderLayout;
4 import java.awt.Color;
5 import java.awt.Dimension;
6 import java.awt.EventQueue;
7 import java.awt.FlowLayout;
8 import java.awt.Font;
9
10 import javax.swing.JLabel;
11 import javax.swing.JOptionPane;
12 import javax.swing.JTextArea;
13 import javax.swing.BorderFactory;
14 import javax.swing.JButton;
15 import javax.swing.JFrame;
16 import javax.swing.JPanel;
17 import javax.swing.JScrollPane;
18 import javax.swing.JTextField;
19 import javax.swing.SwingConstants;
20 import javax.swing.SwingUtilities;
21 import javax.swing.border.EmptyBorder;
```

2. Import `java.awt.*` untuk komponen dasar GUI (layout, warna, font),
import `javax.swing.*` untuk komponen dasar seperti `JFrame`, `JButton`
dan `SwingUtilities` untuk thread safety

```
23 public class InsertionGUI_2511532017 extends JFrame {
24     private static final long serialVersionUID = 1L;
25     private int[] array_2017;
26     private JLabel[] labelArray_2017;
27     private JButton stepButton_2017, resetButton_2017, setButton_2017;
28     private JTextField inputField_2017;
29     private JPanel panelArray_2017;
30     private JTextArea stepArea_2017;
31     private JPanel contentPane_2017;
32
33     private int i_2017 = 1, j_2017;
34     private boolean sorting_2017 = false;
35     private int stepCount_2017 = 1;
36
37 }
```

3. Deklarasi kelas `array_2017` untuk menyimpan data integer yang akan diurutkan, `labelArray_2017` untuk menampilkan setiap elemen secara visual. `sorting_2017` true jika proses sorting sedang aktif. `stepCount_2017` untuk menghitung langkah, dan `i_2017`, `j_2017` sebagai penunjuk indeks untuk insertion sort

```
public InsertionGUI_2511532017() {
    setTitle ("Insertion Sort Langkah Per Langkah");
    setSize(750, 400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout (new BorderLayout());
}
```

4. Constructor inialisasi GUI, ukuran window, `BorderLayout` untuk menata komponen

```
//panel input
JPanel inputPanel_2017 = new JPanel (new FlowLayout());
inputField_2017 = new JTextField(30);
setButton_2017 = new JButton ("Set Array");
inputPanel_2017.add(new JLabel("Masukkan angka (pisahkan dengan koma:"));
inputPanel_2017.add(inputField_2017);
inputPanel_2017.add(setButton_2017);
```

5. Panel input, pengguna memasukkan angka yang dipisahkan dengan koma, dan tombol set Array akan memproses input tersebut

```
//Panel Array visual
panelArray_2017 = new JPanel();
panelArray_2017.setLayout(new FlowLayout());
```

6. Area kosong yang nantinya akan diisi `JLabel` untuk menampilkan elemen array sebagai kotak-kotak visual

```
//panel kontrol
JPanel controlPanel_2017 = new JPanel();
stepButton_2017 = new JButton ("langkah selanjutnya");
resetButton_2017 = new JButton ("reset");
stepButton_2017.setEnabled(false);
controlPanel_2017.add(stepButton_2017);
controlPanel_2017.add(resetButton_2017);
```

7. Panel kontrol, tempat untuk button step dan button reset

```
//area teks untuk log langkah-langkah
stepArea_2017 = new JTextArea (8, 60);
stepArea_2017.setEditable(false);
stepArea_2017.setFont(new Font("Monospaced", Font.PLAIN,14));
JScrollPane scrollPane_2017 = new JScrollPane (stepArea_2017);

//tambahkan panel ke frame
add(inputPanel_2017, BorderLayout.NORTH);
add(panelArray_2017, BorderLayout.CENTER);
add(controlPanel_2017, BorderLayout.SOUTH);
add(scrollPane_2017, BorderLayout.EAST);
```

8. Area log langkah, JScrollPane memungkinkan scrolling jika log terlalu panjang

```
//event set array
setButton_2017.addActionListener(e -> setArrayFromInput_2017());

//event langkah selanjutnya
stepButton_2017.addActionListener(e -> performStep_2017());

//event reset
resetButton_2017.addActionListener(e -> reset_2017());
```

9. Event handler, menggunakan lambda expression untuk memproses klik tombol dengan memanggil method yang sesuai

```
private void setArrayFromInput_2017() {
    String text = inputField_2017.getText().trim();
    if (text.isEmpty()) return;
    String[] parts = text.split(",");
    array_2017 = new int [parts.length];
    try {
        for (int k = 0; k < parts.length; k++) {
            array_2017[k] = Integer.parseInt(parts[k].trim());
        }
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan "+ "dengan koma ", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
}

i_2017 = 1;
stepCount_2017 = 1;
sorting_2017 = true;
stepButton_2017.setEnabled(true);
stepArea_2017.setText("");
panelArray_2017.removeAll();
labelArray_2017 = new JLabel [array_2017.length];
for (int k_2017 = 0; k_2017 < array_2017.length; k_2017++) {
    labelArray_2017[k_2017] = new JLabel (String.valueOf(array_2017[k_2017]));
    labelArray_2017[k_2017].setFont(new Font ("Arial", Font.BOLD,24));
    labelArray_2017[k_2017].setBorder(BorderFactory.createLineBorder(Color.BLACK));
    labelArray_2017[k_2017].setPreferredSize(new Dimension (50,50));
    labelArray_2017[k_2017].setHorizontalAlignment(SwingConstants.CENTER);
    panelArray_2017.add(labelArray_2017[k_2017]);
}
panelArray_2017.revalidate();
panelArray_2017.repaint();
}
```

10. Method `setArrayFromInput_2017()`, untuk memproses input yang akan dipecah menggunakan `split` dan error handling jika input bukan berupa angka. Reset state `i_2017` dikembalikan ke 1 dan `stepCount` ke 1, revalidate dan repaint memaksa panel memperbarui tampilan setelah komponen ditambahkan

```
private void performStep_2017() {
    if (i_2017 < array_2017.length && sorting_2017) {
        int key_2017 = array_2017[i_2017];
        j_2017 = i_2017 - 1;

        StringBuilder stepLog_2017 = new StringBuilder();
        stepLog_2017.append("Langkah ").append(stepCount_2017).append(" : Memasukkan ").append(key_2017).append("\n");
    }
}
```

11. Method `performStep_2017()`, hanya jalan ketika (`i_2017 < length`) dan status sorting aktif, simpan key dan catatan ke log

```
while (j_2017 >= 0 && array_2017[j_2017] > key_2017) {
    array_2017[j_2017 + 1] = array_2017[j_2017];
    j_2017--;
}
array_2017[j_2017 + 1] = key_2017;
```

12. Logika insertion satu iterasi

```
updateLabels_2017();
stepLog_2017.append("Hasil: ").append(arrayToString_2017(array_2017)).append("\n\n");
stepArea_2017.append(stepLog_2017.toString());

i_2017++;
stepCount_2017++;

if (i_2017 == array_2017.length) {
    sorting_2017 = false;
    stepButton_2017.setEnabled(false);
    JOptionPane.showMessageDialog(this, "Sorting selesai");
}
}
```

13. Update tampilan log, `updateLabels_2017()` memperbarui teks di semua `JLabel`, tampilan hasil array akan dicatat di log, cek selesai jika semua elemen sudah di proses

```
public void updateLabels_2017() {
    for (int k_2017 = 0; k_2017 < array_2017.length; k_2017++) {
        labelArray_2017[k_2017].setText(String.valueOf(array_2017[k_2017]));
    }
}
```

14. Method pendukung untuk sinkronisasi nilai array dengan tampilan visual

```

private void reset_2017() {
    inputField_2017.setText("");
    panelArray_2017.removeAll();
    panelArray_2017.revalidate();
    panelArray_2017.repaint();
    stepArea_2017.setText("");
    stepButton_2017.setEnabled(false);
    sorting_2017 = false;
    i_2017 = 1;
    stepCount_2017 = 1;
}

```

15. reset_2017() mengembalikan semua komponen ke state awal

```

private String arrayToString_2017 (int[] arr_2017) {
    StringBuilder sb_2017 = new StringBuilder();
    for (int k_2017 = 0; k_2017 < arr_2017.length; k_2017++) {
        sb_2017.append(arr_2017[k_2017]);
        if (k_2017 < arr_2017.length - 1) sb_2017.append(", ");
    }
    return sb_2017.toString();
}

```

16. arrayToString_2017() mengkonversi array integer ke string format

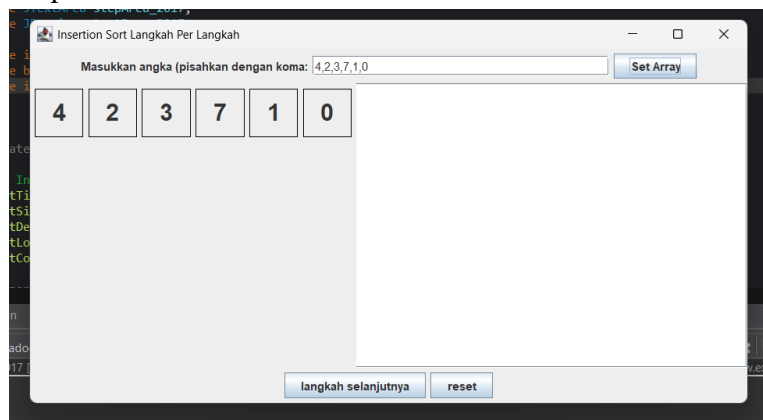
```

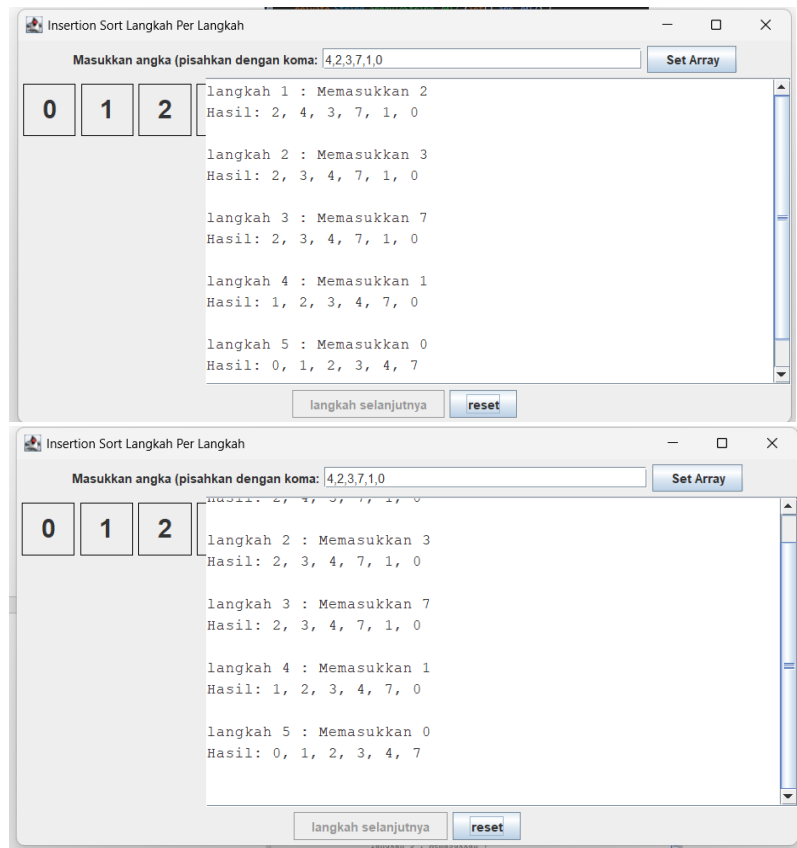
public static void main(String[] args) {
    SwingUtilities.invokeLater(()-> {
        InsertionGUI_2511532017 gui_2017 = new InsertionGUI_2511532017();
        gui_2017.setVisible (true);
    });
}

```

17. method main, SwingUtilities.invokeLater() menjamin GUI dibuat dan dijalankan di event Dispatch Thread (EDT)

18. output





Latihan

- MahasiswaSort_2511532017
 1. Kita akan membuat class baru untuk MahasiswaSort_2511532017

```
package pekan7_2511532017;

public class MahasiswaSort_2511532017 {
    private String nim_2017;
    private String nama_2017;
    private String prodi_2017;
    MahasiswaSort_2511532017(String nim_2017, String nama_2017, String prodi_2017){
        this.nim_2017 = nim_2017;
        this.nama_2017 = nama_2017;
        this.prodi_2017 = prodi_2017;
    }
}
```

2. Deklarasi class dan atribut, private String nim_2017 untuk menyimpan nim, nama_2017 untuk menyimpan nama mahasiswa, prodi_2017 untuk menyimpan prodi dari mahasiswa


```

package pekan7_2511532017;

public class MahasiswaSort_2511532017 {
    private String nim_2017;
    private String nama_2017;
    private String prodi_2017;
    MahasiswaSort_2511532017(String nim_2017, String nama_2017, String prodi_2017){
        this.nim_2017 = nim_2017;
        this.nama_2017 = nama_2017;
        this.prodi_2017 = prodi_2017;
    }
}

```

3. Constructor, method khusus yang dipanggil saat membuat objek MahasiswaSort_2511532017 baru, menginisialisasi judul dan penyanyi

```

1 }
2 public String Nim_2017() {return nim_2017;}
3 public void Nim_2017(String nim_2017) {this.nim_2017 = nim_2017;}
4
5 public String Nama_2017() {return nama_2017;}
6 public void Nama_2017(String nama_2017) {this.nama_2017 = nama_2017;}
7
8 public String Prodi_2017() {return prodi_2017;}
9 public void Prodi_2017(String prodi_2017) {this.prodi_2017 = prodi_2017;}
10
11
12 public String toString() {
13     return "\nNIM : " + nim_2017 +
14           "\nNama : " + nama_2017 +
15           "\nProdi: " + prodi_2017;
16 }
17 }

```

4. Getter untuk mengambil nilai atribut private diluar kelas dan setter untuk mengubah nilai atribut private di luar kelas

- GuiUtama_2511532017

1. Kita buat kelas untuk GuiUtama_2511532017

```

1 package pekan7_2511532017;
2
3 import java.awt.BorderLayout;
4 import java.awt.Color;
5 import java.awt.Dimension;
6 import java.awt.FlowLayout;
7 import java.awt.Font;
8 import java.awt.GridLayout;
9 import java.util.ArrayList;
10
11 import javax.swing.BorderFactory;
12 import javax.swing.JButton;
13 import javax.swing.JComboBox;
14 import javax.swing.JFrame;
15 import javax.swing.JLabel;
16 import javax.swing.JOptionPane;
17 import javax.swing.JPanel;
18 import javax.swing.JScrollPane;
19 import javax.swing.JTextArea;
20 import javax.swing.JTextField;
21 import javax.swing.SwingConstants;
22 import javax.swing.SwingUtilities;
23

```

2. Import.java.awt untuk tata letak, warna, font dan dimensi, javax.swing untuk komponen gui seperti JFrame, JButton, JLabel, dll dan java.util.ArrayList untuk menyimpan data mahasiswa

```
public class GuiUtama_2511532017 extends JFrame {
    private static final long serialVersionUID = 1L;

    private ArrayList<MahasiswaSort_2511532017> listMahasiswa_2017 = new ArrayList<>();
    private ArrayList<MahasiswaSort_2511532017> dataSorting_2017 = new ArrayList<>();

    private JTextField fieldNama_2017, fieldNim_2017, fieldProdi_2017;
    private JComboBox<String> comboAlgo_2017;
    private JPanel panelArray_2017;
    private JLabel[] labelArray_2017;
    private JTextArea stepArea_2017;
    private JButton stepButton_2017, resetButton_2017, mulaiButton_2017;

    private int stepI_2017 = 1;
    private int stepCount_2017 = 1;
    private boolean sorting_2017 = false;
    private String algoMode_2017 = "";
    private int selMinIdx_2017 = 0;
```

3. Deklarasi kelas dan atribut, listMahasiswa_2017 untuk menyimpan data mahasiswa, dataSorting_2017 untuk menyimpan salinan data yang akan disorting, comboAlgo_2017 untuk memilih algoritma sorting yang akan dipakai, algoMode_2017 untuk menyimpan algoritma yang dipilih user, sorting_2017 flag status apakah sorting sedang berjalan, selMinIdx_2017 variabel untuk selectionSort

```
public GuiUtama_2511532017() {
    setTitle("Sorting Mahasiswa");
    setSize(850, 550);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    getContentPane().setLayout(new BorderLayout());

    // Panel input NORTH
    JPanel inputPanel_2017 = new JPanel(new FlowLayout());
    inputPanel_2017.add(new JLabel("Nama:"));
    fieldNama_2017 = new JTextField(10);
    inputPanel_2017.add(fieldNama_2017);
    inputPanel_2017.add(new JLabel("NIM:"));
    fieldNim_2017 = new JTextField(10);
    inputPanel_2017.add(fieldNim_2017);
    inputPanel_2017.add(new JLabel("Prodi:"));
    fieldProdi_2017 = new JTextField(8);
    inputPanel_2017.add(fieldProdi_2017);
    JButton btnTambah_2017 = new JButton("Tambah");
    inputPanel_2017.add(btnTambah_2017);
```

4. Panel input untuk menginputkan data mahasiswa dan tombol tambah

```
//Visualisasi sorting
panelArray_2017 = new JPanel(new GridLayout(0, 3, 5, 5));
panelArray_2017.setBorder(BorderFactory.createTitledBorder("Visualisasi Sorting"));
JScrollPane scrollArray_2017 = new JScrollPane(panelArray_2017);
scrollArray_2017.setPreferredSize(new Dimension(300, 300));
```

5. Panel visualisasi dari inputan user

```
//Panel kontrol
JPanel controlPanel_2017 = new JPanel(new FlowLayout());
comboAlgo_2017 = new JComboBox<>(new String[]{"Insertion Sort", "Selection Sort", "Bubble Sort"});
mulaiButton_2017 = new JButton("Mulai Sorting");
stepButton_2017 = new JButton("Langkah Selanjutnya");
resetButton_2017 = new JButton("Reset");
stepButton_2017.setEnabled(false);
controlPanel_2017.add(comboAlgo_2017);
controlPanel_2017.add(mulaiButton_2017);
controlPanel_2017.add(stepButton_2017);
controlPanel_2017.add(resetButton_2017);
```

6. Panel kontrol untuk mengatur langkah dan reset

```
//Area langkah
stepArea_2017 = new JTextArea(8, 40);
stepArea_2017.setEditable(false);
stepArea_2017.setFont(new Font("Monospaced", Font.PLAIN, 13));
JScrollPane scrollPane_2017 = new JScrollPane(stepArea_2017);

getContentPane().add(inputPanel_2017, BorderLayout.NORTH);
getContentPane().add(scrollArray_2017, BorderLayout.WEST);
getContentPane().add(controlPanel_2017, BorderLayout.SOUTH);
getContentPane().add(scrollPane_2017, BorderLayout.CENTER);

btnTambah_2017.addActionListener(e -> tambahMahasiswa_2017());
mulaiButton_2017.addActionListener(e -> mulaiSorting_2017());
stepButton_2017.addActionListener(e -> performStep_2017());
resetButton_2017.addActionListener(e -> reset_2017());
}
```

7. Area log, area dimana untuk menampilkan detail langkah algoritma

```
//Area langkah
stepArea_2017 = new JTextArea(8, 40);
stepArea_2017.setEditable(false);
stepArea_2017.setFont(new Font("Monospaced", Font.PLAIN, 13));
JScrollPane scrollPane_2017 = new JScrollPane(stepArea_2017);

getContentPane().add(inputPanel_2017, BorderLayout.NORTH);
getContentPane().add(scrollArray_2017, BorderLayout.WEST);
getContentPane().add(controlPanel_2017, BorderLayout.SOUTH);
getContentPane().add(scrollPane_2017, BorderLayout.CENTER);

btnTambah_2017.addActionListener(e -> tambahMahasiswa_2017());
mulaiButton_2017.addActionListener(e -> mulaiSorting_2017());
stepButton_2017.addActionListener(e -> performStep_2017());
resetButton_2017.addActionListener(e -> reset_2017());
}
```

8. Komponen dan event listener, dan menggunakan lambda expression untuk menangani event klik tombol

```
private void tambahMahasiswa_2017() {
    String nama_2017 = fieldNama_2017.getText().trim();
    String nim_2017 = fieldNim_2017.getText().trim();
    String prodi_2017 = fieldProdi_2017.getText().trim();

    if (nama_2017.isEmpty() || nim_2017.isEmpty() || prodi_2017.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Semua field harus diisi!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    listMahasiswa_2017.add(new MahasiswaSort_2511532017(nim_2017, nama_2017, prodi_2017));
    updateLabels_2017();

    fieldNama_2017.setText("");
    fieldNim_2017.setText("");
    fieldProdi_2017.setText("");
}
```

9. Method input data, validasi tidak ada field kosong, MahasiswaSort_2511532017 instance dibuat dan menambahkan ke listMahasiswa_2017 dan UI di update dengan memanggil updateLabels_2017()

```

private void updateLabels_2017() {
    panelArray_2017.removeAll();
    ArrayList<MahasiswaSort_2511532017> src_2017 =
        dataSorting_2017.isEmpty() ? listMahasiswa_2017 : dataSorting_2017;
    labelArray_2017 = new JLabel[src_2017.size()];

    for (int k_2017 = 0; k_2017 < src_2017.size(); k_2017++) {
        labelArray_2017[k_2017] = new JLabel(src_2017.get(k_2017).Nama_2017());
        labelArray_2017[k_2017].setFont(new Font("Arial", Font.BOLD, 15));
        labelArray_2017[k_2017].setBorder(BorderFactory.createLineBorder(Color.BLACK));
        labelArray_2017[k_2017].setPreferredSize(new Dimension(80, 40));
        labelArray_2017[k_2017].setHorizontalAlignment(SwingConstants.CENTER);
        panelArray_2017.add(labelArray_2017[k_2017]);
    }
    panelArray_2017.revalidate();
    panelArray_2017.repaint();
}

```

10. Method untuk update tampilan, jika dataSorting_2017 kosong maka tampilkan listMahasiswa_2017, jika sorting sedang berjalan tampilkan dataSorting_2017, dan menambahkannya ke panel, revalidate() dan repaint() memaksa GUI memperbarui tampilan

```

private void mulaiSorting_2017() {
    if (listMahasiswa_2017.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Data masih kosong!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    dataSorting_2017 = salinList_2017();
    algoMode_2017 = (String) comboAlgo_2017.getSelectedItem();
    stepI_2017 = 1;
    stepCount_2017 = 1;
    selMinIdx_2017 = 0;
    sorting_2017 = true;
    stepButton_2017.setEnabled(true);
    mulaiButton_2017.setEnabled(false);
    stepArea_2017.setText("=== " + algoMode_2017.toUpperCase() + " ===\n");
    stepArea_2017.append("Data awal:\n" + toStringList_2017(dataSorting_2017) + "\n\n");
    updateLabels_2017();
}

```

11. Memulai sorting, salinList_2017() digunakan agar jika user mereset, data asli tetap utuh

```

private void performStep_2017() {
    if (!sorting_2017) return;
    if ("Insertion Sort".equals(algoMode_2017)) stepInsertion_2017();
    else if ("Selection Sort".equals(algoMode_2017)) stepSelection_2017();
    else stepBubble_2017();
}

private void stepInsertion_2017() {
    if (stepI_2017 >= dataSorting_2017.size()) { selesai_2017(); return; }

    MahasiswaSort_2511532017 key_2017 = dataSorting_2017.get(stepI_2017);
    int j_2017 = stepI_2017 - 1;
    while (j_2017 >= 0 &&
        dataSorting_2017.get(j_2017).Nama_2017().compareToIgnoreCase(key_2017.Nama_2017()) > 0) {
        dataSorting_2017.set(j_2017 + 1, dataSorting_2017.get(j_2017));
        j_2017--;
    }
    dataSorting_2017.set(j_2017 + 1, key_2017);

    stepArea_2017.append("Langkah " + stepCount_2017 + " : Memasukkan [" + key_2017.Nama_2017() + "]\n");
    stepArea_2017.append("Hasil:\n" + toStringList_2017(dataSorting_2017) + "\n\n");

    updateLabels_2017();
    stepI_2017++;
    stepCount_2017++;
    if (stepI_2017 >= dataSorting_2017.size()) selesai_2017();
}

```

12. Eksekusi langkah perlangkah, memanggil algoritma berdasarkan algoritma yang dipilih user

```

private void stepInsertion_2017() {
    if (stepI_2017 >= dataSorting_2017.size()) { selesai_2017(); return; }

    MahasiswaSort_2511532017 key_2017 = dataSorting_2017.get(stepI_2017);
    int j_2017 = stepI_2017 - 1;
    while (j_2017 >= 0 &&
        dataSorting_2017.get(j_2017).Nama_2017().compareToIgnoreCase(key_2017.Nama_2017()) > 0) {
        dataSorting_2017.set(j_2017 + 1, dataSorting_2017.get(j_2017));
        j_2017--;
    }
    dataSorting_2017.set(j_2017 + 1, key_2017);

    stepArea_2017.append("Langkah " + stepCount_2017 + " : Memasukkan [" + key_2017.Nama_2017() + "]\n");
    stepArea_2017.append("Hasil:\n" + toStringList_2017(dataSorting_2017) + "\n\n");

    updateLabels_2017();
    stepI_2017++;
    stepCount_2017++;
    if (stepI_2017 >= dataSorting_2017.size()) selesai_2017();
}

```

13. `stepInsertion_2017()`, mengambil elemen (key), lalu menggeser elemen di sebelah kirinya yang lebih besar kemudian menyisipkan key di posisi yang tepat. Menggunakan `compareToIgnoreCase()` berdasarkan nama

```

private void stepSelection_2017() {
    int n_2017 = dataSorting_2017.size();

    if (stepI_2017 >= n_2017 - 1) {
        selesai_2017();
        return;
    }
    selMinIdx_2017 = stepI_2017;
    for (int j_2017 = stepI_2017 + 1; j_2017 < n_2017; j_2017++) {
        if (dataSorting_2017.get(j_2017).Nama_2017()
            .compareToIgnoreCase(dataSorting_2017.get(selMinIdx_2017).Nama_2017()) < 0) {
            selMinIdx_2017 = j_2017;
        }
    }

    MahasiswaSort_2511532017 temp_2017 = dataSorting_2017.get(selMinIdx_2017);
    dataSorting_2017.set(selMinIdx_2017, dataSorting_2017.get(stepI_2017));
    dataSorting_2017.set(stepI_2017, temp_2017);

    stepArea_2017.append("Pass " + stepCount_2017 + " : Min = [" + temp_2017.Nama_2017() +
        "]" -> posisi " + stepI_2017 + "\n");
    stepArea_2017.append("Hasil:\n" + toStringList_2017(dataSorting_2017) + "\n\n");

    updateLabels_2017();
    stepI_2017++;
    stepCount_2017++;
    if (stepI_2017 >= n_2017 - 1) selesai_2017();
}

```

14. `stepSelection_2017()`, setiap langkah mencari nilai minimum dari bagian array yang belum terurut, lalu menukarnya ke posisi paling depan dari bagian tersebut

```

private void stepBubble_2017() {
    int n_2017 = dataSorting_2017.size();

    if (stepI_2017 >= n_2017) {
        selesai_2017();
        return;
    }

    boolean swapped_2017 = false;

    for (int j_2017 = 0; j_2017 < n_2017 - stepI_2017; j_2017++) {
        if (dataSorting_2017.get(j_2017).Nama_2017()
            .compareToIgnoreCase(dataSorting_2017.get(j_2017 + 1).Nama_2017()) > 0) {
            MahasiswaSort_2511532017 temp_2017 = dataSorting_2017.get(j_2017);
            dataSorting_2017.set(j_2017, dataSorting_2017.get(j_2017 + 1));
            dataSorting_2017.set(j_2017 + 1, temp_2017);
            swapped_2017 = true;
        }
    }

    stepArea_2017.append("Pass " + stepCount_2017 + " : \n" + toStringList_2017(dataSorting_2017) + "\n\n");
    updateLabels_2017();
    stepI_2017++;
    stepCount_2017++;

    if (!swapped_2017 || stepI_2017 >= n_2017) {
        selesai_2017();
    }
}

```

15. stepBubble_2017(), elemen besar pindah (mengapung) ke kanan melalui serangkaian penukaran berdekatan, swapped_2017 memungkinkan algoritma berhenti lebih awal jika data sudah terurut sebelum iterasi.

```

private void selesai_2017() {
    sorting_2017 = false;
    stepButton_2017.setEnabled(false);
    stepArea_2017.append("=== Sorting selesai! ===\n");
    stepArea_2017.append("Hasil akhir:\n" + toStringList_2017(dataSorting_2017) + "\n");
    JOptionPane.showMessageDialog(this, "Sorting selesai!");
}

```

16. Method selesai_2017() menandai proses akhir, menampilkan hasil akhir dan memberikan notifikasi

```

private String toStringList_2017(ArrayList<MahasiswaSort_2511532017> list_2017) {
    StringBuilder sb_2017 = new StringBuilder();
    for (int i_2017 = 0; i_2017 < list_2017.size(); i_2017++) {
        sb_2017.append("---- Mahasiswa ").append(i_2017 + 1).append(" ----");
        sb_2017.append(list_2017.get(i_2017).toString());
        if (i_2017 < list_2017.size() - 1) sb_2017.append("\n");
    }
    return sb_2017.toString();
}

private ArrayList<MahasiswaSort_2511532017> salinList_2017() {
    ArrayList<MahasiswaSort_2511532017> copy_2017 = new ArrayList<>();
    for (MahasiswaSort_2511532017 m : listMahasiswa_2017) {
        copy_2017.add(new MahasiswaSort_2511532017(m.Nim_2017(), m>Nama_2017(), m.Prodi_2017()));
    }
    return copy_2017;
}

```

17. toStringList_2017() mengonversi ArrayList<Mahasiswa> menjadi string untuk ditampilkan JTextArea

```

private String toStringList_2017(ArrayList<MahasiswaSort_2511532017> list_2017) {
    StringBuilder sb_2017 = new StringBuilder();
    for (int i_2017 = 0; i_2017 < list_2017.size(); i_2017++) {
        sb_2017.append("---- Mahasiswa ").append(i_2017 + 1).append(" ----");
        sb_2017.append(list_2017.get(i_2017).toString());
        if (i_2017 < list_2017.size() - 1) sb_2017.append("\n");
    }
    return sb_2017.toString();
}

private ArrayList<MahasiswaSort_2511532017> salinList_2017() {
    ArrayList<MahasiswaSort_2511532017> copy_2017 = new ArrayList<>();
    for (MahasiswaSort_2511532017 m : listMahasiswa_2017) {
        copy_2017.add(new MahasiswaSort_2511532017(m.Nim_2017(), m>Nama_2017(), m.Prodi_2017()));
    }
    return copy_2017;
}

```

18. salinList_2017(), memcegah modifikasi data asli

```

private void reset_2017() {
    listMahasiswa_2017.clear();
    dataSorting_2017.clear();
    panelArray_2017.removeAll();
    panelArray_2017.revalidate();
    panelArray_2017.repaint();
    stepArea_2017.setText("");
    stepButton_2017.setEnabled(false);
    mulaiButton_2017.setEnabled(true);
    sorting_2017 = false;
    stepI_2017 = 1;
    stepCount_2017 = 1;
    fieldNama_2017.setText("");
    fieldNim_2017.setText("");
    fieldProdi_2017.setText("");
}

```

19. reset_2017(), membersihkan semua data dan mengembalikan GUI ke keadaan awal

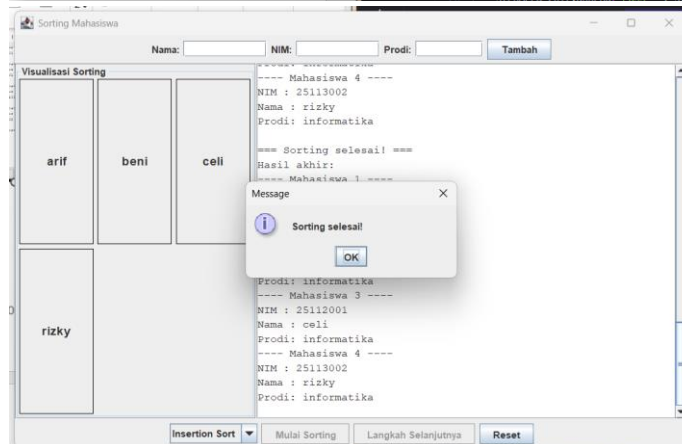
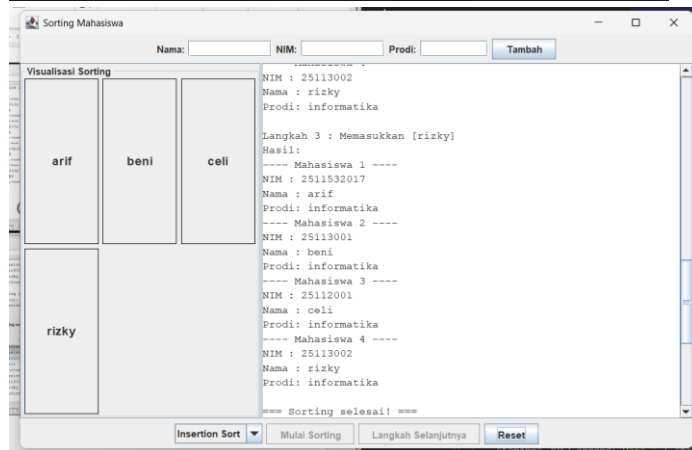
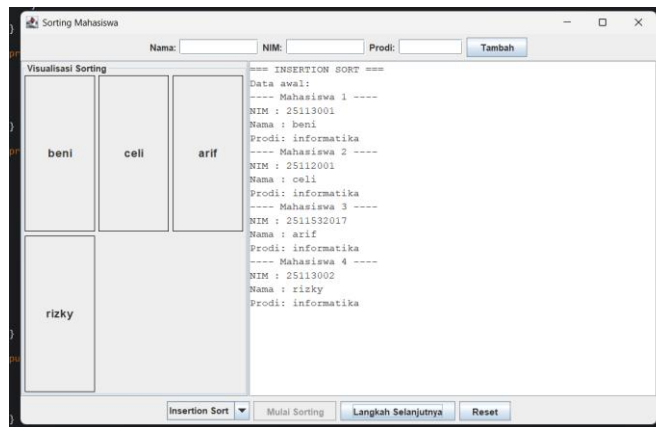
```

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        GuiUtama_2511532017 gui_2017 = new GuiUtama_2511532017();
        gui_2017.setVisible(true);
    });
}

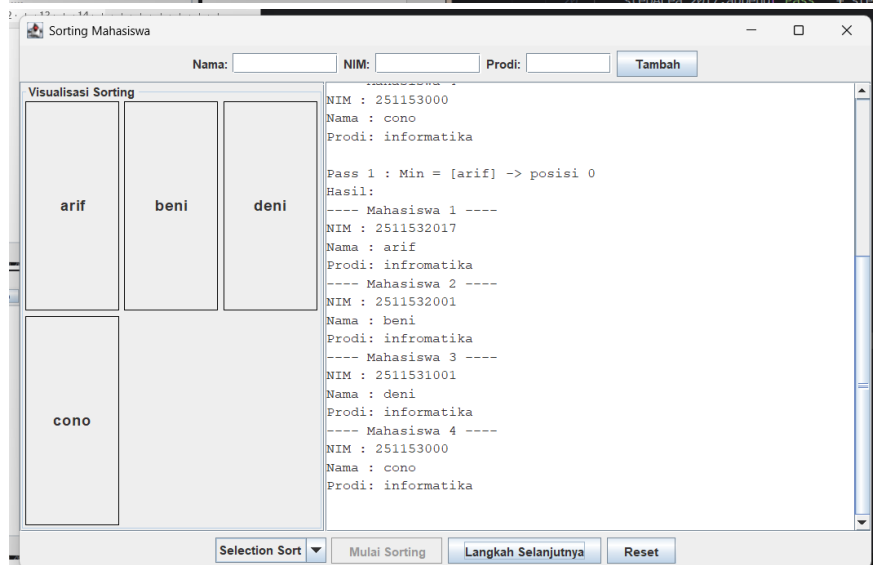
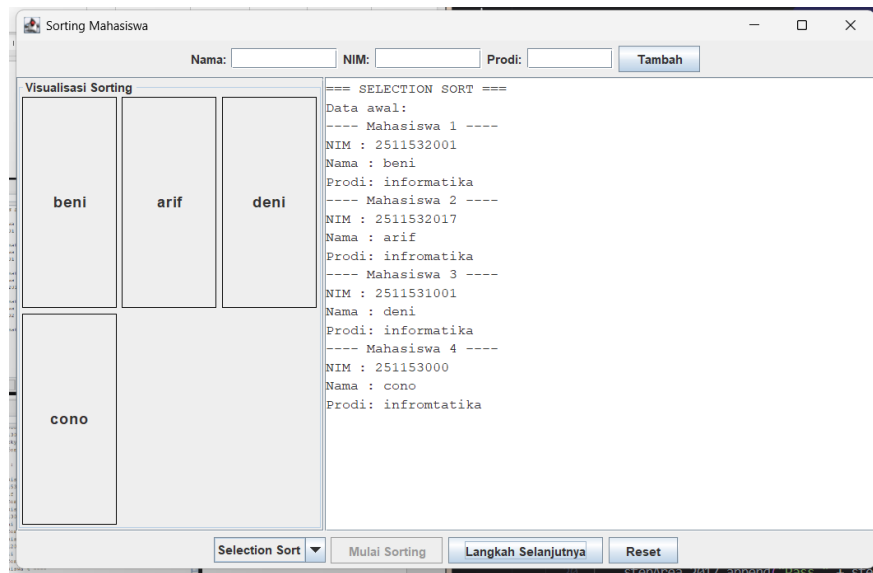
```

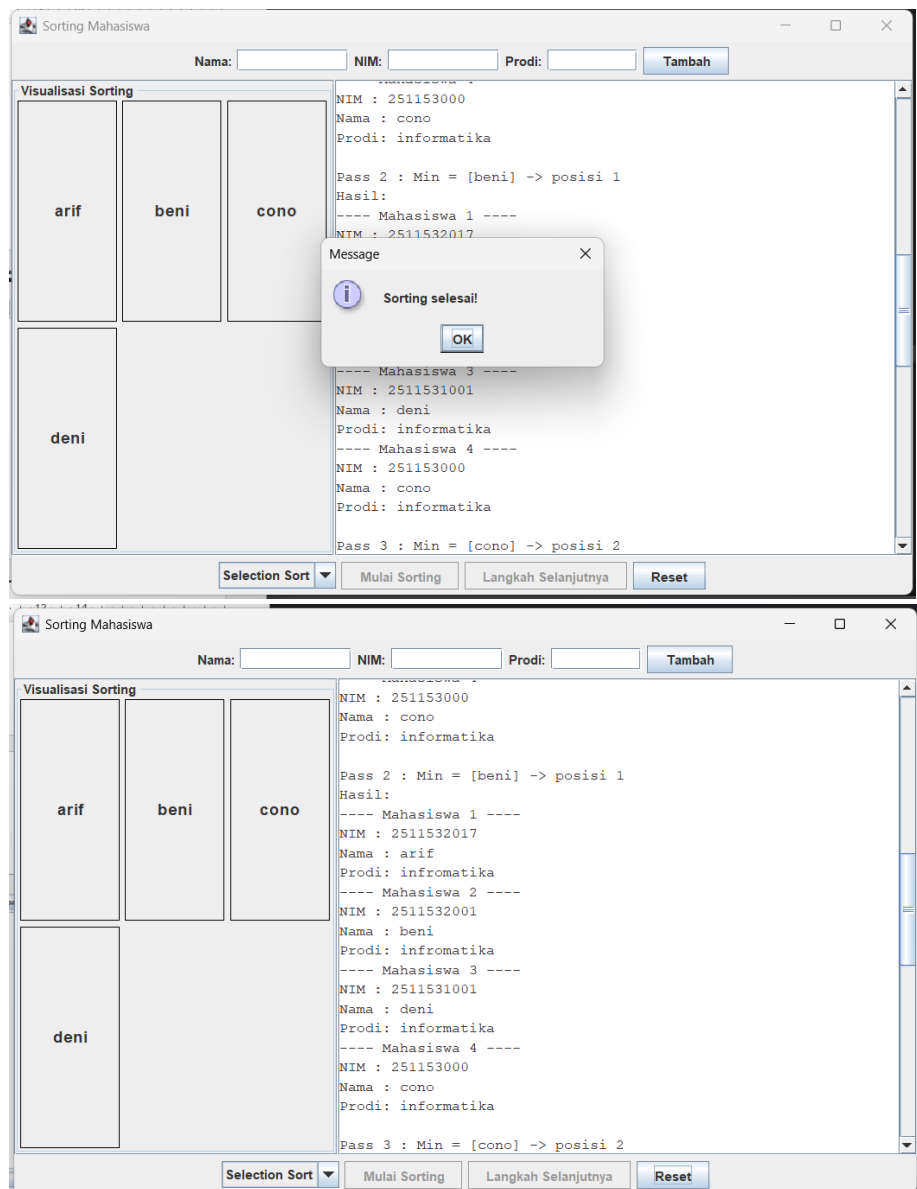
20. method main, memastikan pembuatan dan penampilan GUI dilakukan di EDT

21. Output
InsertionSort

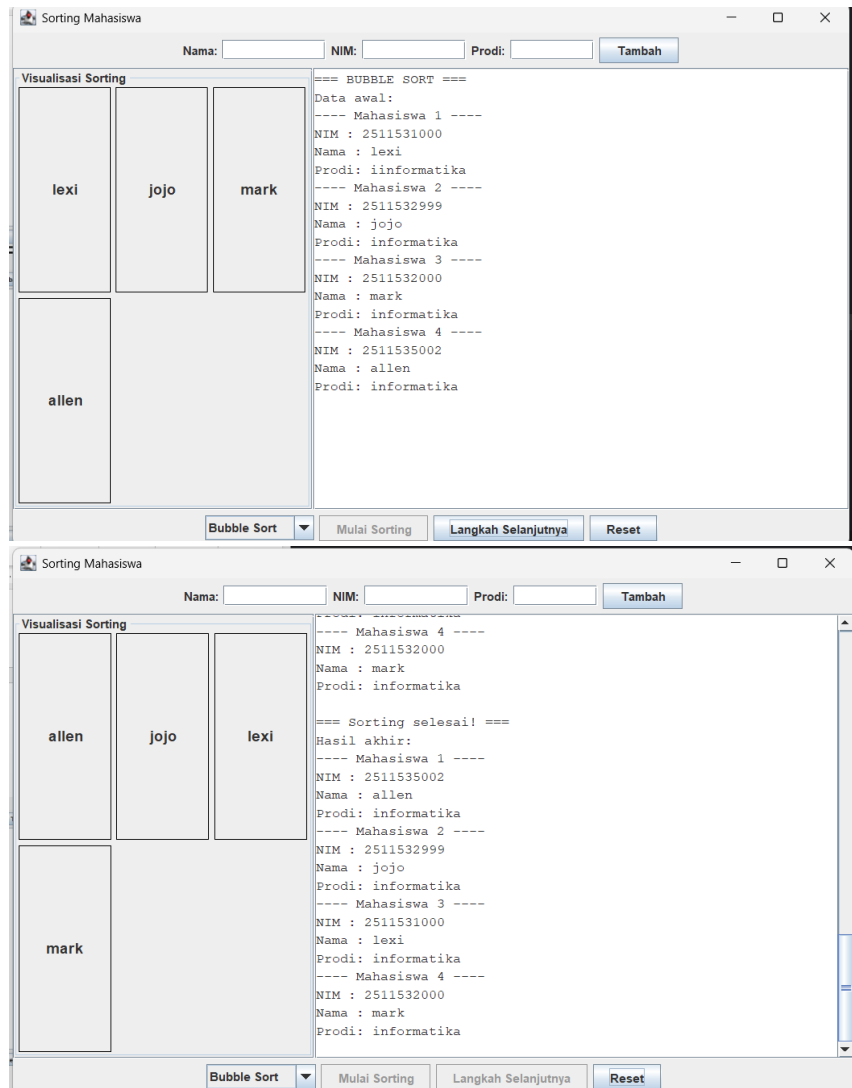


SelectionSort

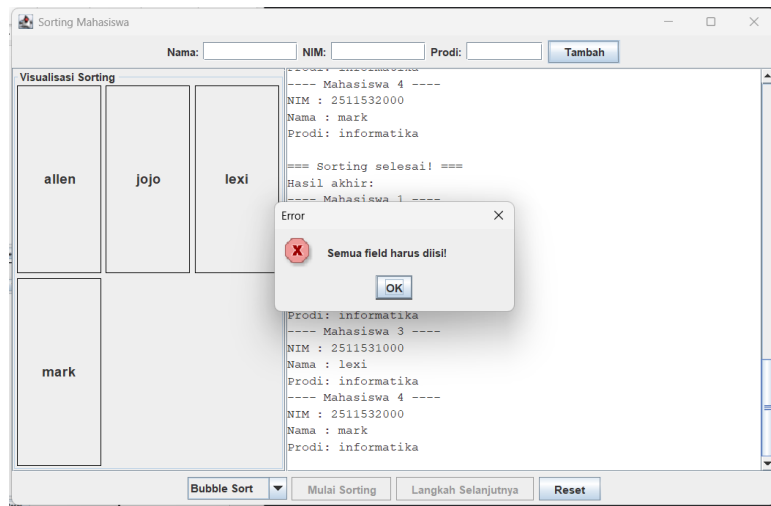




BubbleSort



Error message



BAB III

KESIMPULAN

Berdasarkan kesimpulan yang telah dilaksanakan, dapat disimpulkan bahwa ketiga algoritma sorting sederhana yaitu bubbleSort, selectionSort dan insertionSort berhasil diimplementasikan. Praktikum ini menunjukkan kompleksitas waktu $O(n^2)$ masing-masing memiliki karakteristik yang berbeda Bubble Sort mengandalkan penukaran elemen berdekatan, Selection Sort meminimalkan operasi *swap* dengan mencari elemen minimum setiap iterasi, dan Insertion Sort paling adaptif untuk data yang hampir terurut.

DAFTAR PUSTAKA

[1] GeeksforGeeks, "Sorting Algorithms," GeeksforGeeks, 2026. [Online]. Available: <https://www.geeksforgeeks.org/sorting-algorithms/> [Accessed: May 20, 2026].